

2.1 Introduction

Dans ce chapitre d'analyse des exigences fonctionnelles et de conception, nous aborderons plusieurs elements essentiels pour la comprehension et la definition des besoins du systeme *Test Management Platform*. Nous commencerons par le recueil des besoins fonctionnels, qui englobe les fonctionnalites specifiques que le systeme doit offrir pour repondre aux attentes des utilisateurs. Ensuite, nous nous pencherons sur le recueil des besoins non fonctionnels, qui concernent les contraintes techniques et les performances du systeme.

Nous explorerons egalement l'identification des parties prenantes, en mettant l'accent sur l'identification des acteurs et leurs roles et responsabilites. Enfin, nous aborderons les user stories, les diagrammes de cas d'utilisation (Use Case Diagrams), les diagrammes de sequence, le diagramme de classes et les diagrammes complementaires (activite, etats-transitions, composants, deploiement) pour obtenir une vue globale et claire des exigences fonctionnelles et de la conception du systeme.

2.2 Specification du besoin

2.2.1 Recueil des besoins fonctionnels

Dans cette section dediee au recueil des besoins fonctionnels, nous examinerons en detail les besoins identifies pour notre projet Test Management Platform en tenant compte des problemes et defis auxquels nos utilisateurs potentiels sont confrontes. Nous chercherons a mettre en place des solutions adaptees pour repondre a ces besoins. Bases sur la problematique et les solutions proposees dans le chapitre precedent, voici les principaux besoins fonctionnels que nous avons determines :

- Gestion de l'authentification et de la securite (JWT).
- Gestion des utilisateurs (creation, activation, desactivation, suppression).
- Gestion des projets (creation, modification, archivage, assignation des membres).
- Gestion des modules de test au sein de chaque projet.
- Gestion des scenarios de test au sein de chaque module.
- Gestion des cas de test (creation manuelle, import Excel, generation par IA).

- Gestion des sessions de test (campagnes de test regroupant des executions).
- Gestion des executions de test (lancement, mise a jour du statut, capture d'ecran).
- Gestion des anomalies (declaration, suivi, changement de statut, commentaires).
- Tableau de bord et rapports (dashboard global, KPIs, generation PDF).
- Fonctionnalites d'intelligence artificielle (generation de cas de test, resume de session, suggestion de gravite, chatbot).
- Systeme de notifications automatiques.

Des entretiens ont ete menes avec les responsables du departement qualite logicielle et les futurs utilisateurs de la plateforme, afin de recueillir leurs attentes et contraintes. Un benchmark des solutions existantes (TestRail, Zephyr, qTest) a permis de comparer les pratiques du marche et de reperer les points d'amelioration. Ces differentes approches ont abouti a un ensemble coherent de besoins fonctionnels, en adequation avec les objectifs du projet et les attentes des utilisateurs finaux.

2.2.2 Recueil des besoins non fonctionnels

Dans cette section dediee au recueil des besoins non fonctionnels, nous presentons les exigences qui, bien que ne concernant pas directement les fonctionnalites, jouent un role essentiel dans la qualite globale de la plateforme Test Management Platform. Ces besoins garantissent la securite, la fiabilite et une experience utilisateur optimale.

Voici les besoins non fonctionnels identifies pour notre projet :

- **Securite** : l'acces au systeme doit etre protege par une authentication basee sur JWT (JSON Web Token) avec une duree de validite de 24 heures. Chaque mot de passe est chiffre avec BCrypt. Les autorisations sont verifiees par Spring Security via les annotations `@PreAuthorize`.
- **Performance et temps de reponse** : le systeme doit assurer un temps de reponse inferieur a 2 secondes pour toute action utilisateur courante (authentication, navigation, chargement de contenus).
- **Ergonomie** : l'interface utilisateur doit etre intuitive, responsive et accessible depuis differents navigateurs (Chrome, Firefox, Edge). Le design utilise Tailwind CSS pour garantir une experience cohérente.

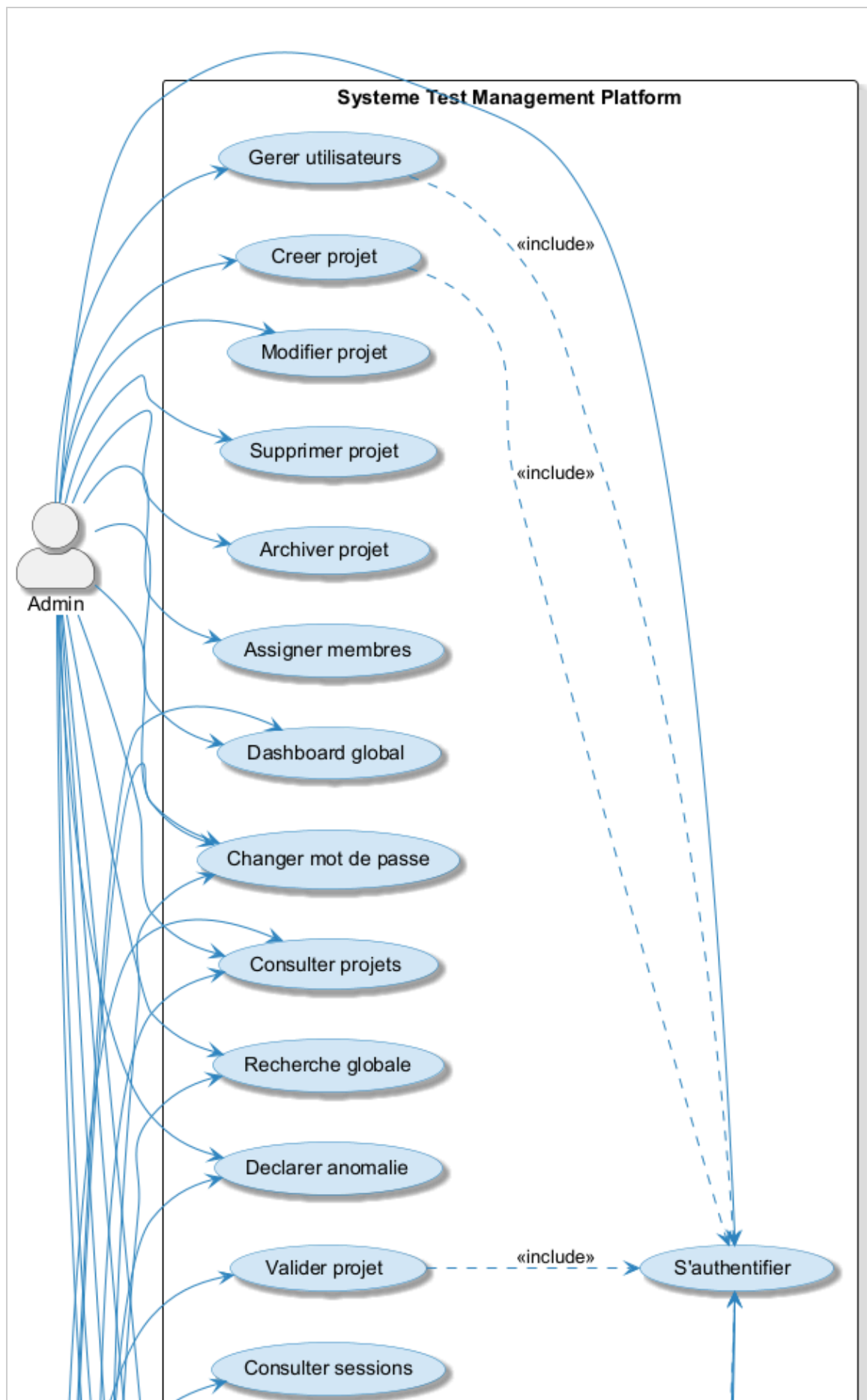
- **Disponibilite** : le systeme doit etre disponible 99% du temps pendant les heures de travail. En cas d'indisponibilite du serveur IA (Ollama), les fonctionnalites classiques doivent rester operationnelles.
- **Maintenabilite** : le code doit suivre une architecture en couches (Controller, Service, Repository) facilitant la maintenance et l'evolution.
- **Extensibilite** : l'architecture modulaire doit permettre l'ajout de nouvelles fonctionnalites sans impacter l'existant.

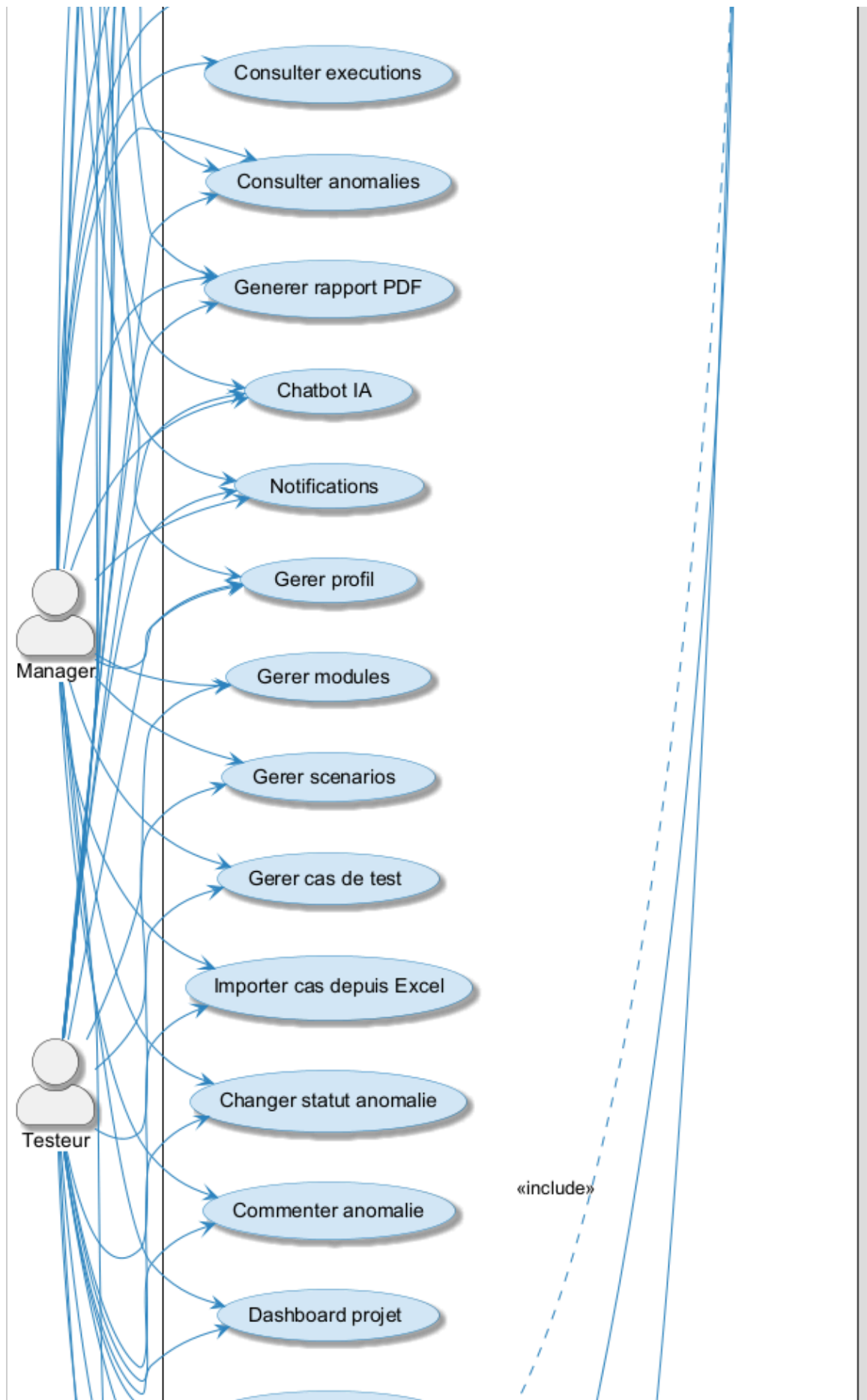
2.2.3 Identification des parties prenantes

i. Identification des acteurs

Les acteurs principaux du systeme Test Management Platform sont :

- **Admin**
- **Manager**
- **Testeur**





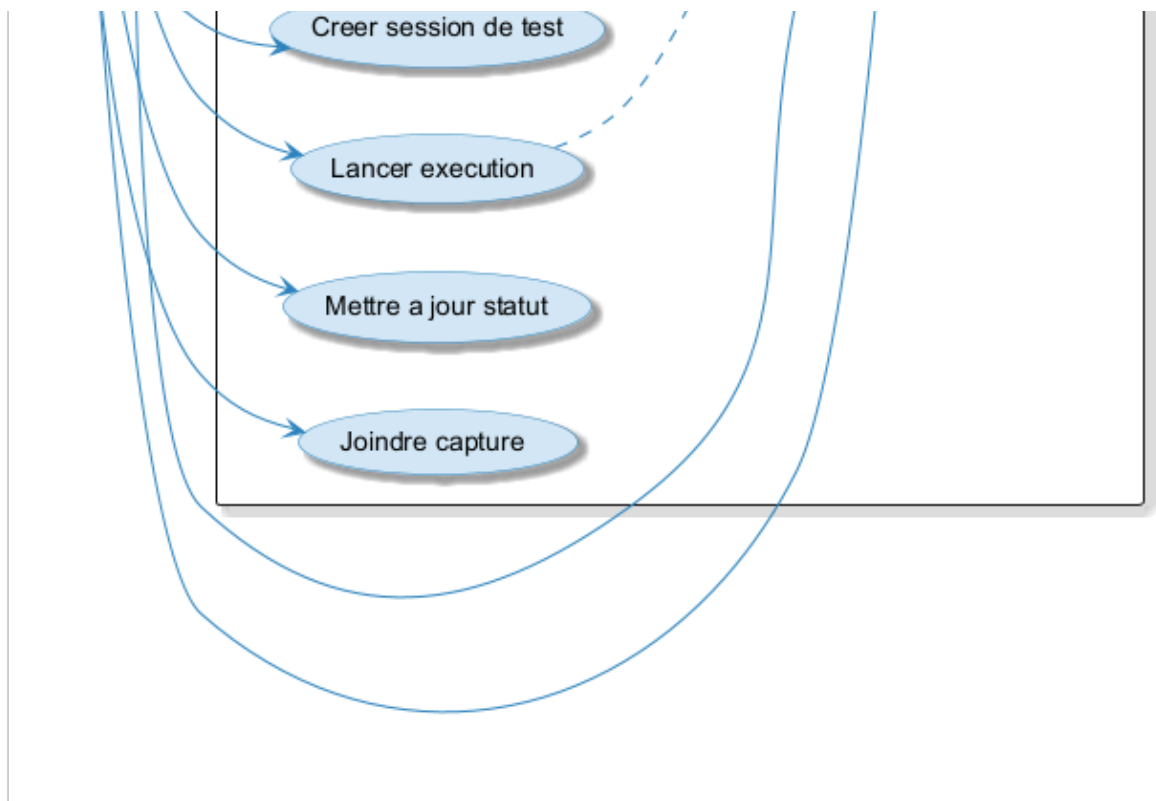


Figure 2-1 - Repartition des roles dans le systeme

ii. Les roles et responsabilites de chaque acteur

Voir Tableau 2-1 : ce tableau presente les roles et responsabilites de chaque acteur de la plateforme.

Acteur	Role	Responsabilites
Admin	Administrateur de la plateforme	- Gerer les comptes utilisateurs (creation, modification, activation, desactivation, suppression). - Creer, modifier, supprimer et archiver les projets. - Assigner et retirer des membres aux projets. - Declarer et supprimer des anomalies. - Consulter le dashboard global et les KPIs globaux. - Acceder aux archives des projets. - Generer des rapports PDF. - Utiliser le chatbot IA et la recherche globale.
Manager	Responsable qualite / Chef de projet test	- Valider les projets apres verification des resultats de test. - Gerer les modules, scenarios et cas de test (creation, modification, suppression). - Importer des cas de test depuis un fichier Excel. - Consulter les sessions de test et les executions (lecture seule). - Changer le statut des anomalies et ajouter des commentaires. - Consulter le dashboard global et par projet. - Generer des rapports PDF.
Testeur	Ingenieur de test / QA	- Gerer les modules, scenarios et cas de test (creation, modification, suppression). - Importer des cas de test depuis Excel ou les generer avec l'IA. - Creer et gerer les sessions de test (campagnes). - Lancer les executions, mettre a jour le statut et joindre des captures d'ecran. - Declarer des anomalies avec suggestion de gravite par IA. - Consulter le dashboard par projet. - Utiliser le chatbot IA.

Tableau 2-1 - Les acteurs de la plateforme

2.2.4 User Stories de la solution proposee

Nous decrivons ci-dessous les fonctionnalites de la plateforme Test Management Platform sous forme de User Stories, en decrivant pour chaque acteur les actions qu'il peut effectuer.

Utilisateur (tous les roles)

- En tant qu'utilisateur, je peux m'authentifier a la plateforme avec un nom d'utilisateur et un mot de passe.
- En tant qu'utilisateur connecte, je peux consulter mes informations personnelles.
- En tant qu'utilisateur connecte, je peux modifier mes informations personnelles.
- En tant qu'utilisateur connecte, je peux changer mon mot de passe.
- En tant qu'utilisateur connecte, je peux recevoir des notifications.
- En tant qu'utilisateur connecte, je peux interroger le chatbot IA.

Admin

- En tant qu'Admin connecte, je peux creer, consulter, modifier, activer, desactiver et supprimer des utilisateurs.
- En tant qu'Admin connecte, je peux lister les testeurs disponibles pour les assigner aux projets.
- En tant qu'Admin connecte, je peux creer, consulter, modifier et supprimer des projets.
- En tant qu'Admin connecte, je peux archiver un projet termine.
- En tant qu'Admin connecte, je peux assigner et retirer des membres (testeurs, managers) a un projet.
- En tant qu'Admin connecte, je peux declarer une anomalie liee a une execution.
- En tant qu'Admin connecte, je peux supprimer une anomalie.
- En tant qu'Admin connecte, je peux consulter le dashboard global avec les KPIs globaux.
- En tant qu'Admin connecte, je peux consulter les archives des projets.
- En tant qu'Admin connecte, je peux generer un rapport PDF.
- En tant qu'Admin connecte, je peux effectuer une recherche globale.

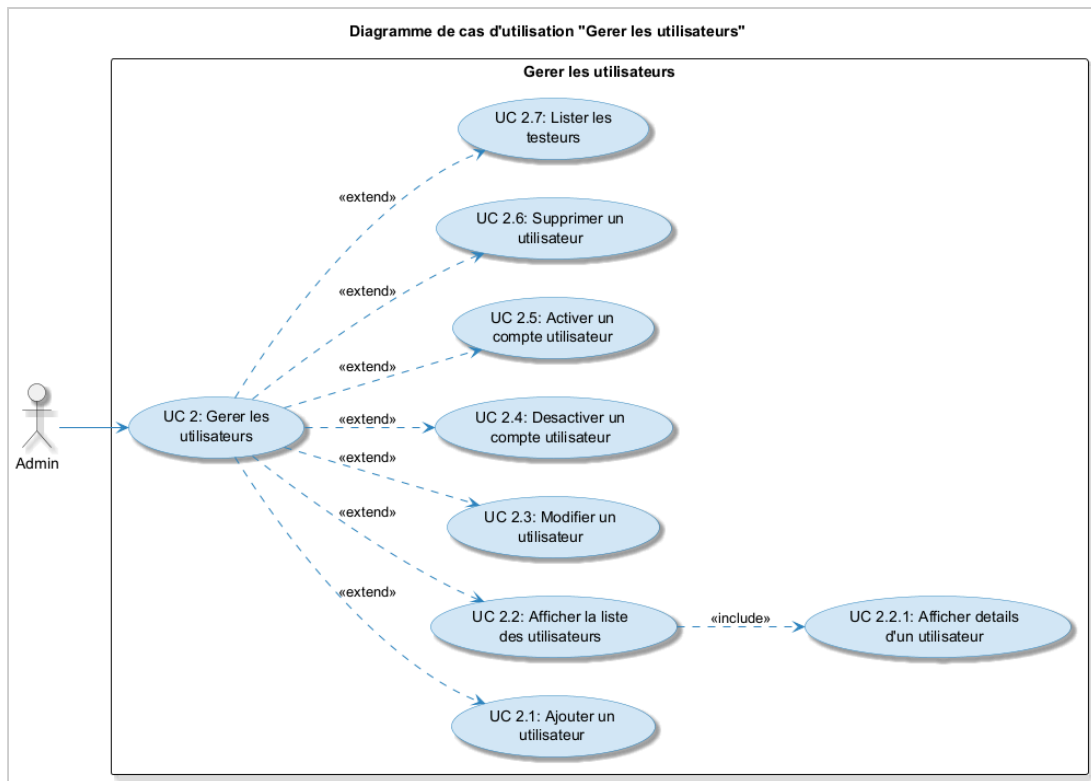


Figure 2-2 - Hierarchie de l'Admin

Manager

- En tant que Manager connecte, je peux consulter les projets auxquels je suis assigne.
- En tant que Manager connecte, je peux valider un projet apres verification des resultats de test.
- En tant que Manager connecte, je peux creer, consulter, modifier et supprimer des modules dans un projet.
- En tant que Manager connecte, je peux creer, consulter, modifier et supprimer des scenarios dans un module.
- En tant que Manager connecte, je peux creer, consulter, modifier et supprimer des cas de test dans un scenario.
- En tant que Manager connecte, je peux importer des cas de test depuis un fichier Excel.
- En tant que Manager connecte, je peux consulter les sessions de test et leurs details.
- En tant que Manager connecte, je peux consulter les executions et leurs details (lecture seule).
- En tant que Manager connecte, je peux consulter les anomalies, changer leur statut et ajouter des commentaires.
- En tant que Manager connecte, je peux consulter le dashboard global et les KPIs par projet.

- En tant que Manager connecte, je peux generer un rapport PDF.

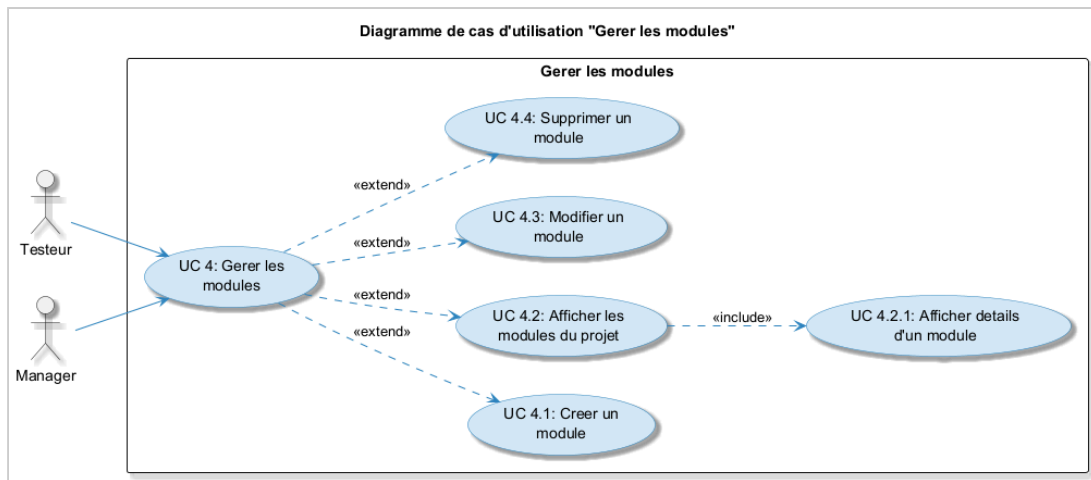


Figure 2-3 - Hierarchie du Manager

Testeur

- En tant que Testeur connecte, je peux consulter les projets auxquels je suis assigne.
- En tant que Testeur connecte, je peux creer, consulter, modifier et supprimer des modules dans un projet.
- En tant que Testeur connecte, je peux creer, consulter, modifier et supprimer des scenarios dans un module.
- En tant que Testeur connecte, je peux creer, consulter, modifier et supprimer des cas de test.
- En tant que Testeur connecte, je peux importer des cas de test depuis un fichier Excel (.xlsx).
- En tant que Testeur connecte, je peux generer des cas de test automatiquement avec l'IA a partir d'une description.
- En tant que Testeur connecte, je peux creer et modifier des sessions de test.
- En tant que Testeur connecte, je peux lancer une execution d'un cas de test dans une session.
- En tant que Testeur connecte, je peux mettre a jour le statut d'une execution (SUCCESS, FAILED, BLOCKED).
- En tant que Testeur connecte, je peux joindre une capture d'ecran a une execution via Cloudinary.
- En tant que Testeur connecte, je peux declarer une anomalie avec suggestion automatique de gravite par l'IA.

- En tant que Testeur connecte, je peux consulter les anomalies, changer leur statut et ajouter des commentaires.
- En tant que Testeur connecte, je peux cloturer une session, ce qui genere un resume IA avec recommandation go/no-go.
- En tant que Testeur connecte, je peux consulter le dashboard par projet et generer un rapport PDF.

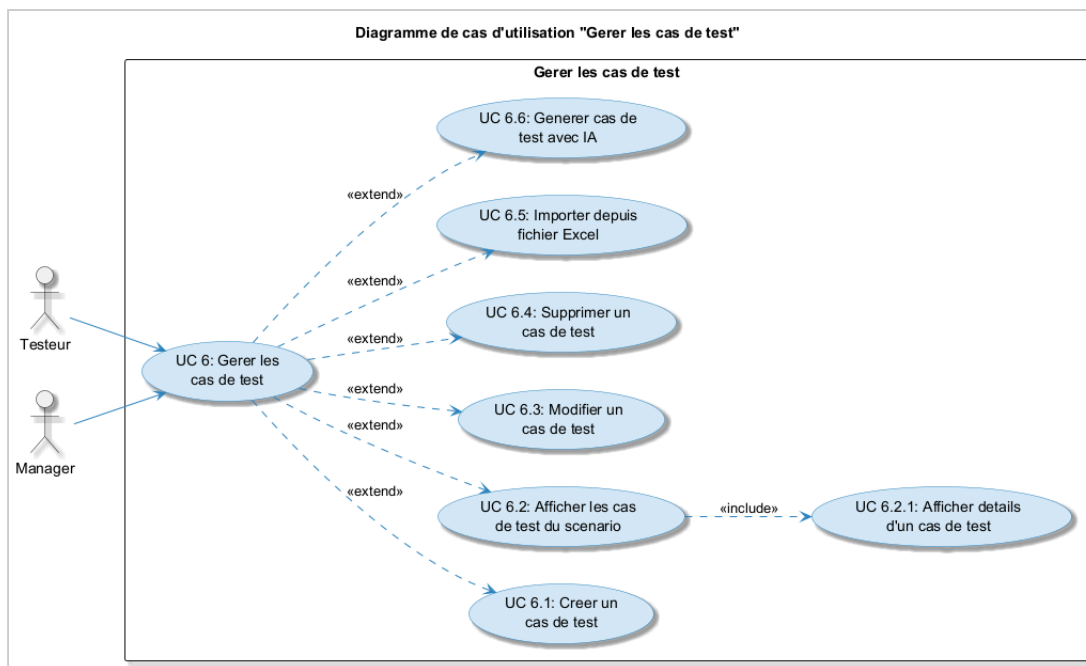


Figure 2-4 - Hierarchie du Testeur

2.3 Langage UML

Le langage UML (Unified Modeling Language, ou langage de modelisation unifie) a ete pense pour etre un langage de modelisation visuelle commun, riche semantiquement et syntaxiquement, destine a l'architecture, la conception et l'implementation de systemes logiciels complexes par leur structure aussi bien que leur comportement. L'UML a des applications qui vont au-dela du developpement logiciel, notamment pour les flux de processus dans l'industrie.

Il ressemble aux plans utilises dans d'autres domaines et se compose de differents types de diagrammes. Dans l'ensemble, les diagrammes UML decrivent la limite, la structure et le comportement du systeme et des objets qui s'y trouvent.

L'UML n'est pas un langage de programmation, mais il existe des outils qui peuvent etre utilises pour generer du code en plusieurs langages a partir de diagrammes UML. L'UML a une relation directe avec l'analyse et la conception orientees objet.

Les differents elements representables sont :

- Activite d'un objet/logiciel
- Acteurs
- Processus
- Schema de base de donnees
- Composants logiciels
- Reutilisation de composants

2.4 Diagrammes UML

2.4.1 Diagrammes de cas d'utilisation

En langage de modelisation unifie (UML), un diagramme de cas d'utilisation peut servir a resumer les informations des utilisateurs de votre systeme (egalement appeles acteurs) et leurs interactions avec ce dernier. La creation de ce type de diagramme UML requiert un ensemble de symboles et de connecteurs specifiques.

Ce cas d'utilisation (Figure 2-5) concerne la gestion du profil par l'ensemble des acteurs du systeme. Chaque utilisateur connecte (Admin, Manager ou Testeur) peut acceder a son profil personnel pour consulter ses informations, les modifier ou changer son mot de passe. Ces trois sous-cas sont lies au cas principal par des relations «extend», car ils sont optionnels et declenches par choix de l'utilisateur.

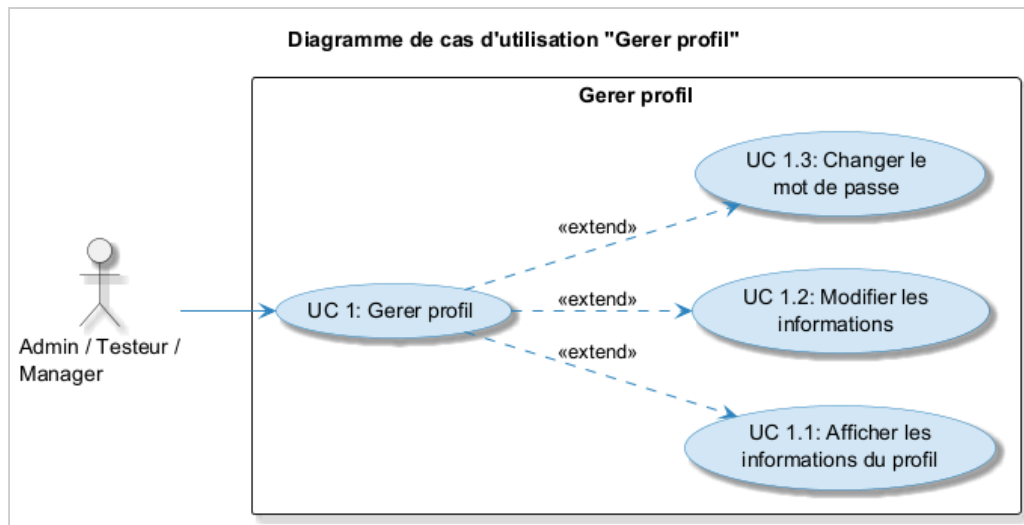


Figure 2-5 - Diagramme de cas d'utilisation : Gerer le profil

Ce cas d'utilisation (Figure 2-6) concerne la gestion des comptes utilisateurs, une fonctionnalite reservee exclusivement a l'Admin. L'application permet a l'administrateur connecte d'effectuer les operations suivantes : ajouter un nouvel utilisateur, lister tous les utilisateurs, modifier les informations d'un compte, desactiver ou activer un compte, supprimer definitivement un utilisateur, et lister les testeurs disponibles pour l'assignation aux projets.

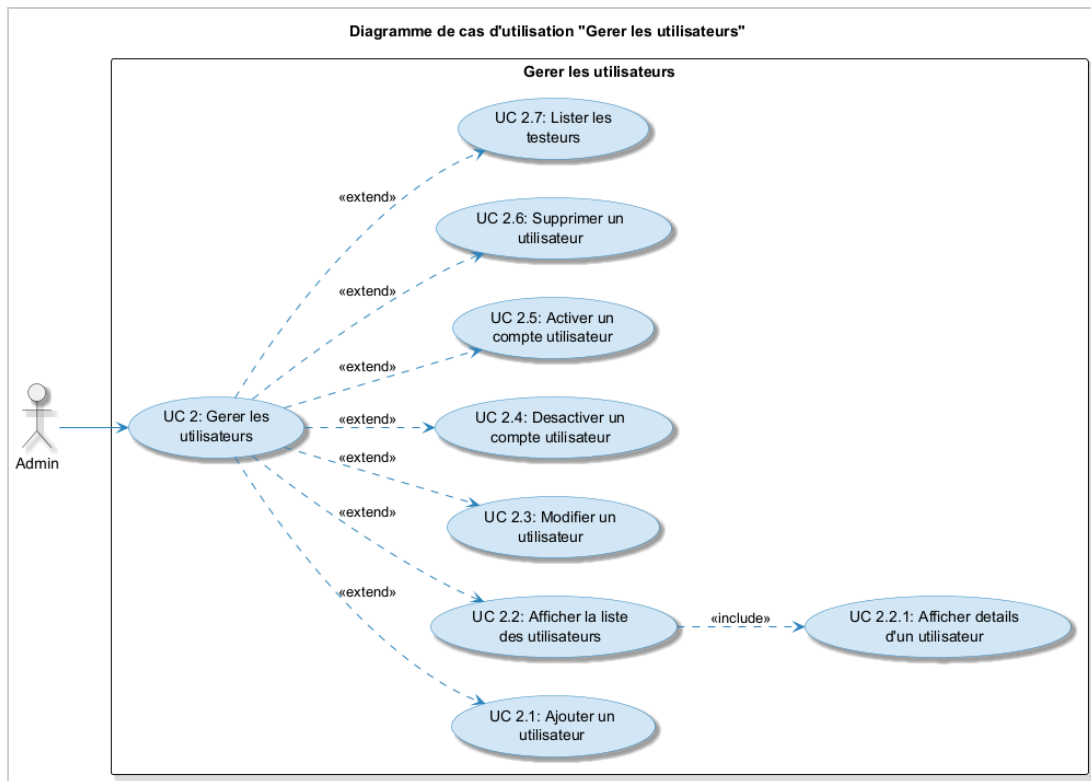


Figure 2-6 - Diagramme de cas d'utilisation : Gerer les utilisateurs

Ce cas d'utilisation (Figure 2-7) concerne la gestion des projets. L'Admin peut creer, modifier, supprimer et archiver des projets, ainsi qu'assigner ou retirer des membres. Le Manager peut uniquement consulter les projets et les valider apres verification des resultats de test. Le Testeur ne peut que consulter les projets auxquels il est assigne.

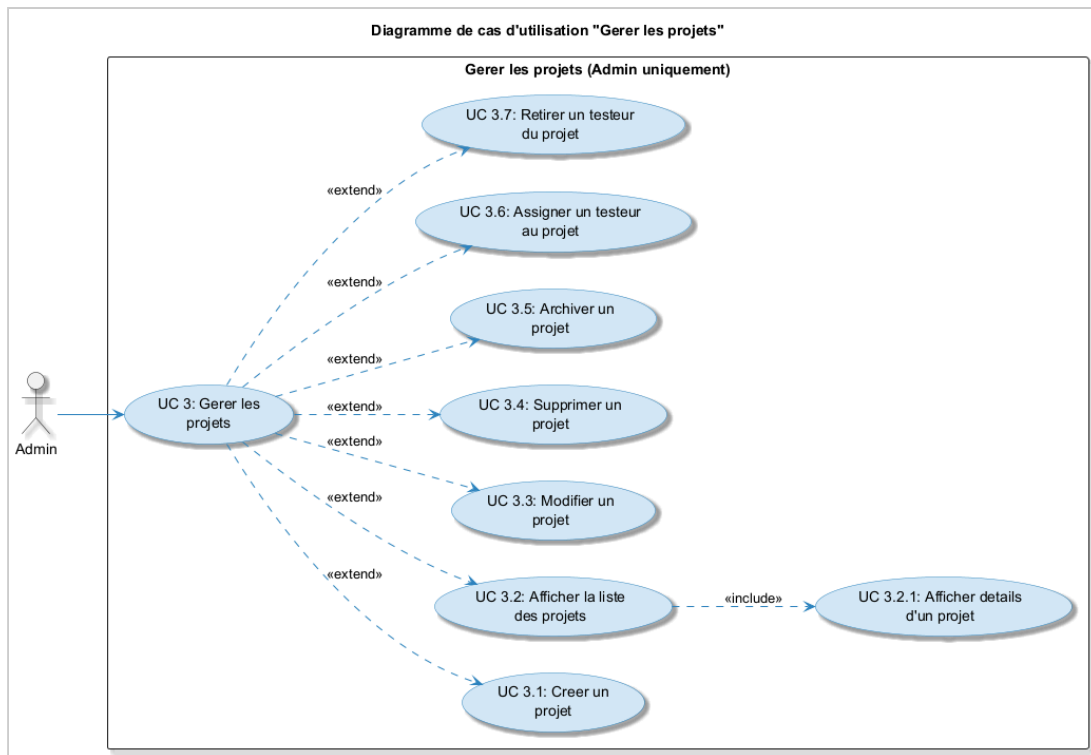


Figure 2-7 - Diagramme de cas d'utilisation : Gerer les projets

Ce cas d'utilisation (Figure 2-8) concerne la gestion des modules au sein d'un projet. Un module represente une composante fonctionnelle du projet a tester (par exemple : "Module Authentification", "Module Paiement"). L'application permet au Testeur et au Manager de creer, consulter, modifier et supprimer des modules. L'Admin n'intervient pas dans cette gestion operationnelle.

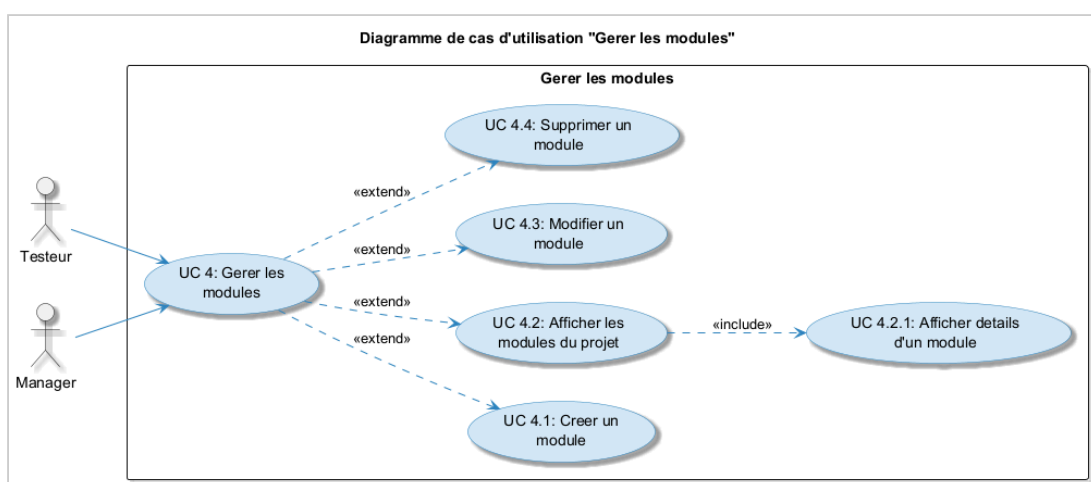


Figure 2-8 - Diagramme de cas d'utilisation : Gerer les modules

Ce cas d'utilisation (Figure 2-9) concerne la gestion des scenarios de test au sein de chaque module. Un scenario represente un parcours fonctionnel a tester. Comme pour les modules, le Testeur et le Manager partagent les memes permissions : creation, consultation, modification et suppression des scenarios. La consultation inclut l'affichage des details d'un scenario specifique.

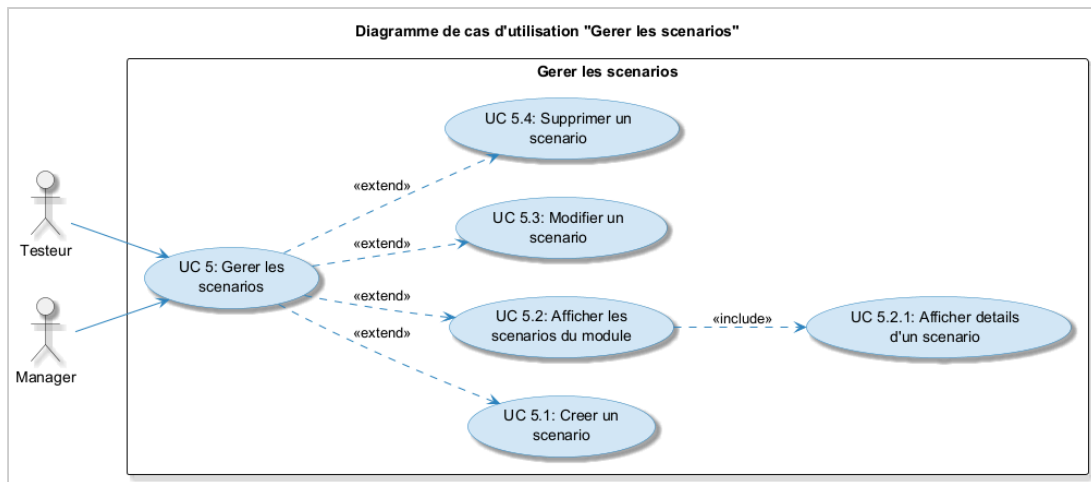


Figure 2-9 - Diagramme de cas d'utilisation : Gerer les scenarios

Ce cas d'utilisation (Figure 2-10) concerne la gestion des cas de test, l'élément central du processus. Chaque cas de test décrit une procédure précise à exécuter avec des données d'entrée, des étapes et un résultat attendu. Le Testeur et le Manager peuvent créer, consulter, modifier et supprimer des cas de test. Deux fonctionnalités additionnelles sont disponibles : l'import en masse depuis un fichier Excel (.xlsx) via Apache POI, et la génération automatique par intelligence artificielle via Ollama.

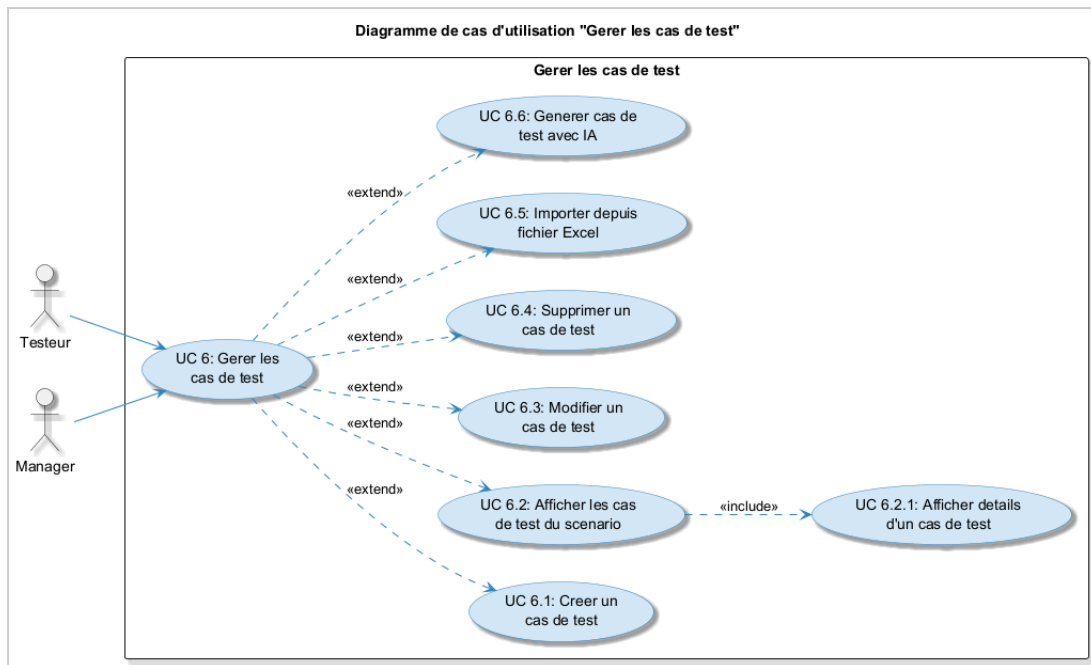


Figure 2-10 - Diagramme de cas d'utilisation : Gerer les cas de test

Ce cas d'utilisation (Figure 2-11) concerne la gestion des sessions de test. Une session regroupe un ensemble d'executions dans un contexte temporel defini, comme une campagne de test pour un sprint. Le Testeur peut créer, modifier et cloturer des sessions. La cloture declenche automatiquement la generation d'un resume IA avec recommandation go/no-go. Le Manager dispose d'un acces en lecture seule pour consulter les sessions et leurs details.

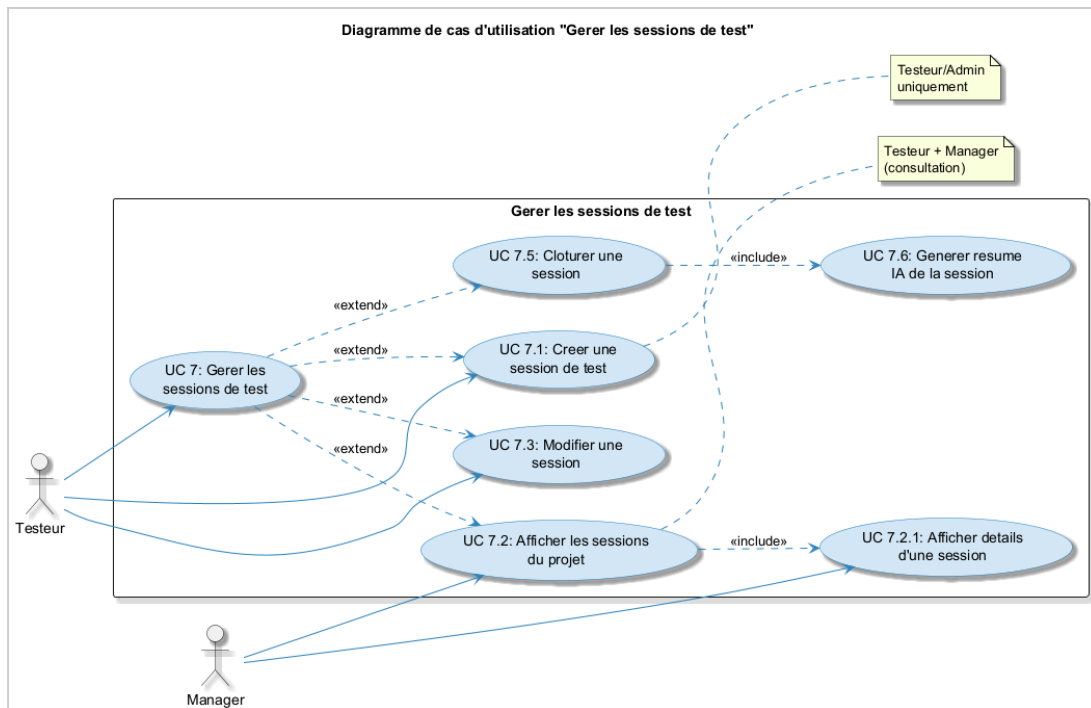


Figure 2-11 - Diagramme de cas d'utilisation : Gerer les sessions de test

Ce cas d'utilisation (Figure 2-12) concerne la gestion des executions de test. Le Testeur lance l'execution d'un cas de test dans le cadre d'une session, met a jour le statut (SUCCESS, FAILED ou BLOCKED) et peut joindre une capture d'ecran comme preuve via Cloudinary. Le Manager ne dispose que d'un acces en consultation aux executions, sans pouvoir en creer ni en modifier.

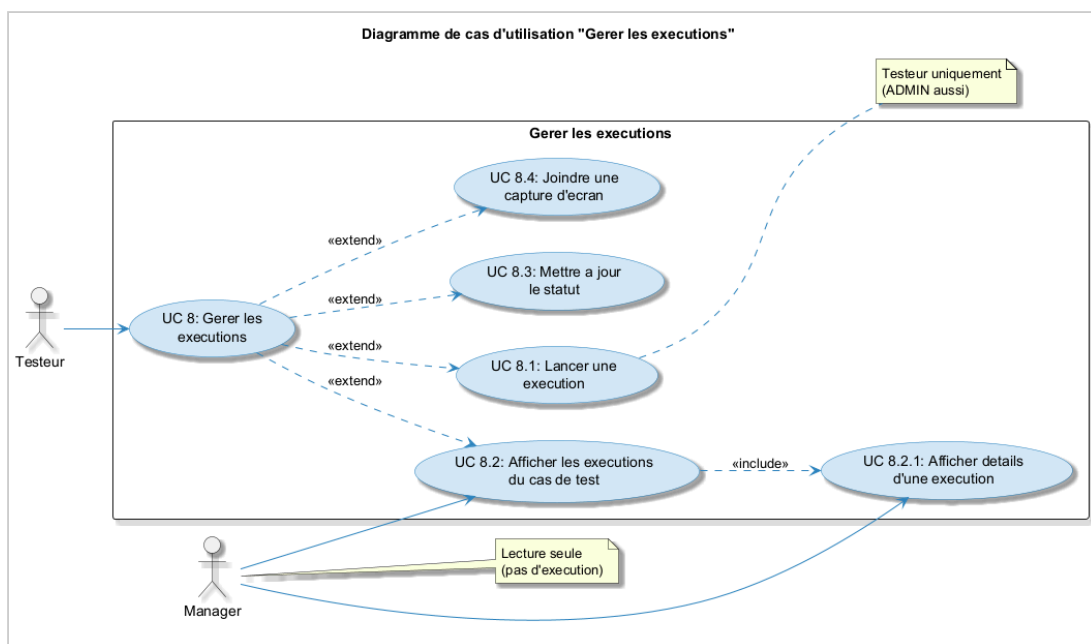


Figure 2-12 - Diagramme de cas d'utilisation : Gerer les executions

Ce cas d'utilisation (Figure 2-13) concerne la gestion des anomalies, un module où chaque rôle a des permissions distinctes. Le Testeur et l'Admin peuvent déclarer des anomalies. La déclaration inclut automatiquement une suggestion de gravité par l'IA («include»). Le Manager ne peut que consulter les anomalies, changer leur statut et ajouter des commentaires. La suppression d'une anomalie est réservée exclusivement à l'Admin.

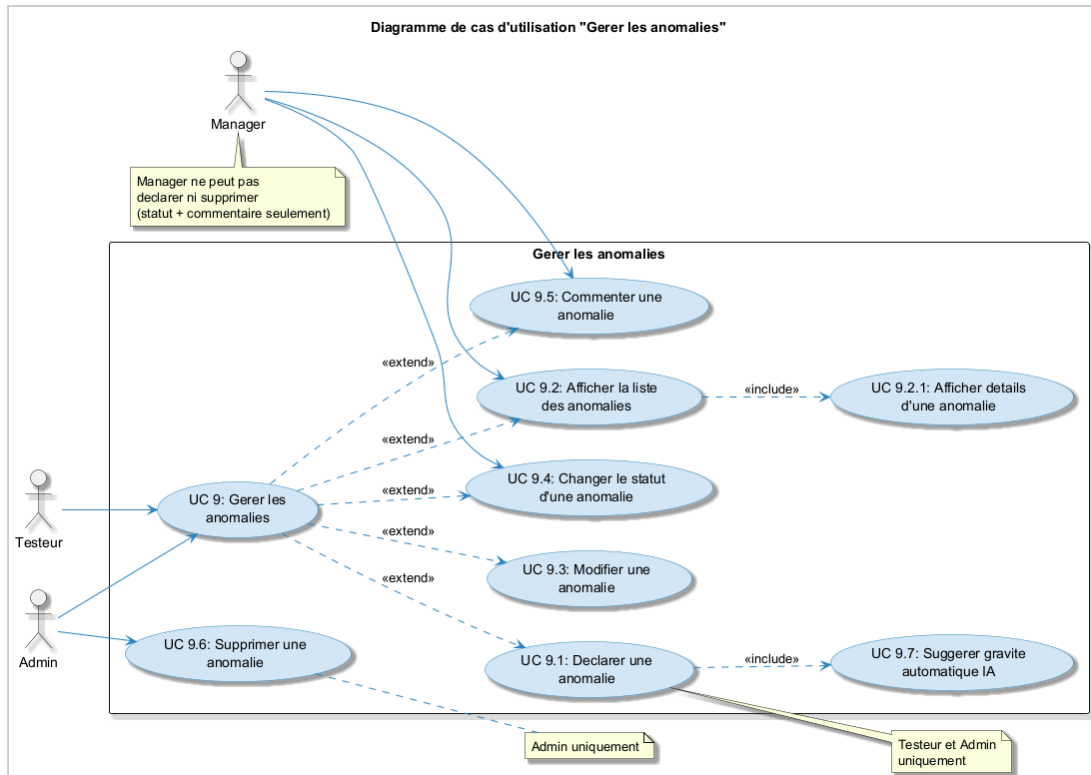


Figure 2-13 - Diagramme de cas d'utilisation : Gerer les anomalies

Ce cas d'utilisation (Figure 2-14) concerne le tableau de bord et les rapports. L'Admin et le Manager accèdent au dashboard global et aux KPIs globaux. Le Testeur n'a accès qu'aux KPIs des projets auxquels il est assigné. L'Admin dispose en plus de l'accès exclusif aux archives. Tous les rôles peuvent générer des rapports PDF et utiliser la recherche.

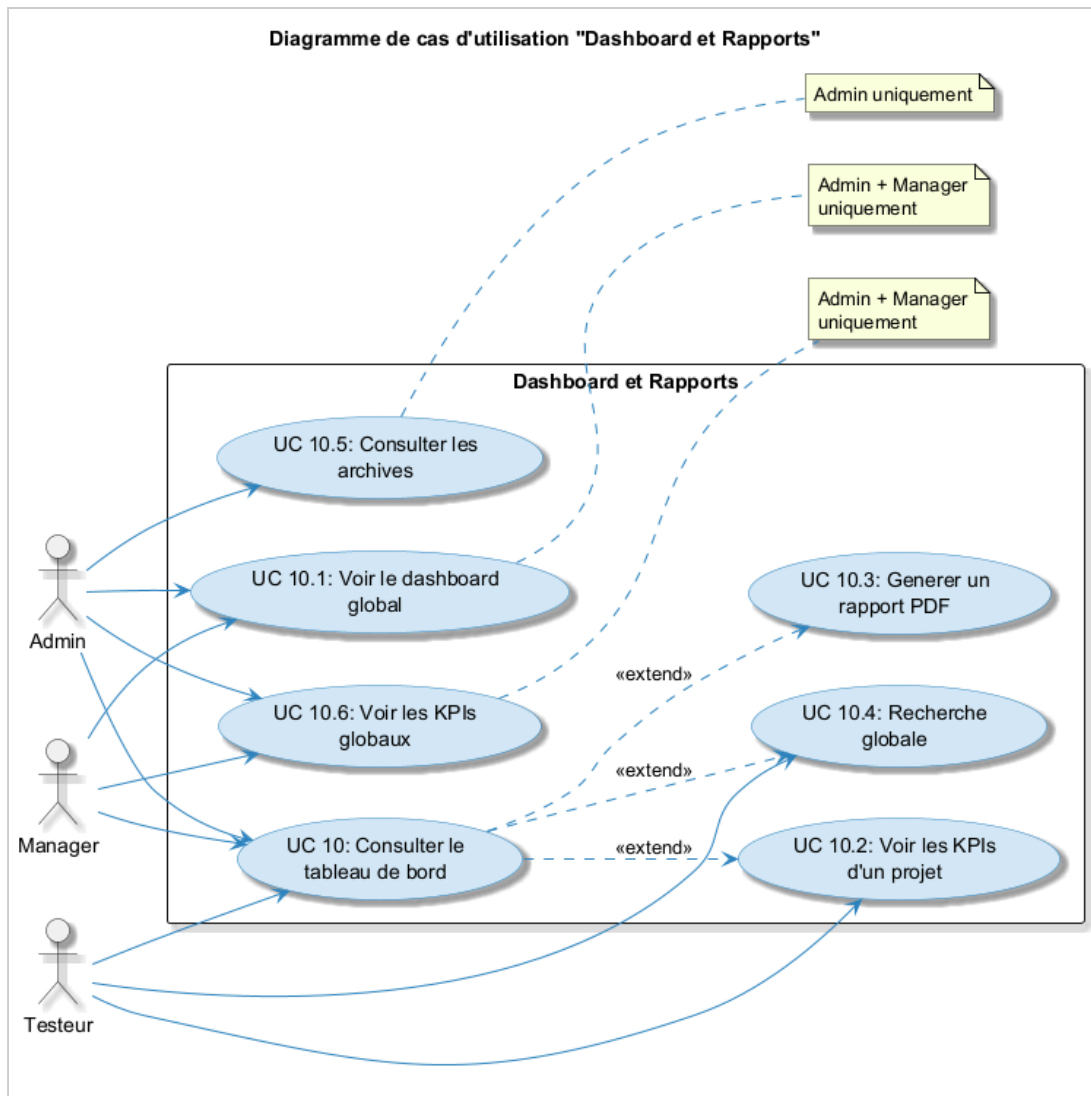


Figure 2-14 - Diagramme de cas d'utilisation : Dashboard et rapports

Ce cas d'utilisation (Figure 2-15) concerne les fonctionnalités d'intelligence artificielle intégrées à la plateforme via Ollama (modèle deepseek-r1:7b). Tous les acteurs peuvent générer des cas de test automatiquement, obtenir un résumé intelligent de session, recevoir des suggestions de gravité d'anomalie et interroger le chatbot IA. Chaque fonctionnalité inclut obligatoirement une vérification de la disponibilité du serveur LLM («include»).

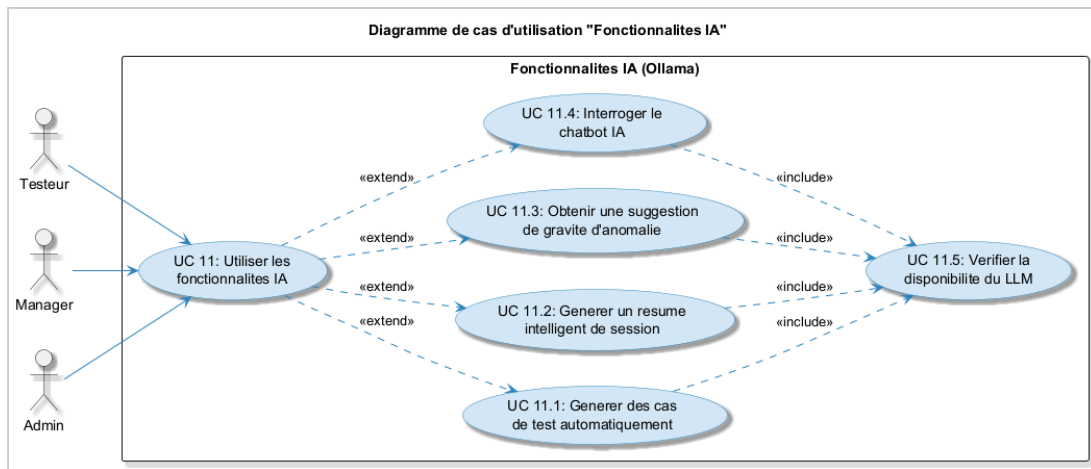


Figure 2-15 - Diagramme de cas d'utilisation : Fonctionnalites IA

2.4.2 Diagrammes de sequence

Un diagramme de sequence est un type de diagramme d'interaction, car il decrit comment et dans quel ordre plusieurs objets fonctionnent ensemble. Ces diagrammes sont utilises a la fois par les developpeurs logiciels et les managers d'entreprises pour analyser les besoins d'un nouveau systeme ou documenter un processus existant. Les diagrammes de sequence sont parfois appeles diagrammes d'evenements ou scenarios d'evenements.

Dans cette partie dediee a la realisation des diagrammes de sequences, nous allons explorer en profondeur cet outil essentiel de l'ingenierie logicielle.

- Diagramme de sequence d'authentification :

Cette Figure 2-16 represente le processus d'authentification d'un utilisateur. L'acteur saisit son nom d'utilisateur et son mot de passe dans le formulaire de connexion. Le frontend envoie une requete POST a /api/auth/login. Le AuthController delegue au AuthService qui verifie les identifiants en base de donnees via BCrypt. En cas de succes, un token JWT valable 24h est genere par le JwtService et renvoye au frontend, qui le stocke dans le localStorage et redirige vers le dashboard. En cas d'echec, une erreur 401 Unauthorized est retournee.

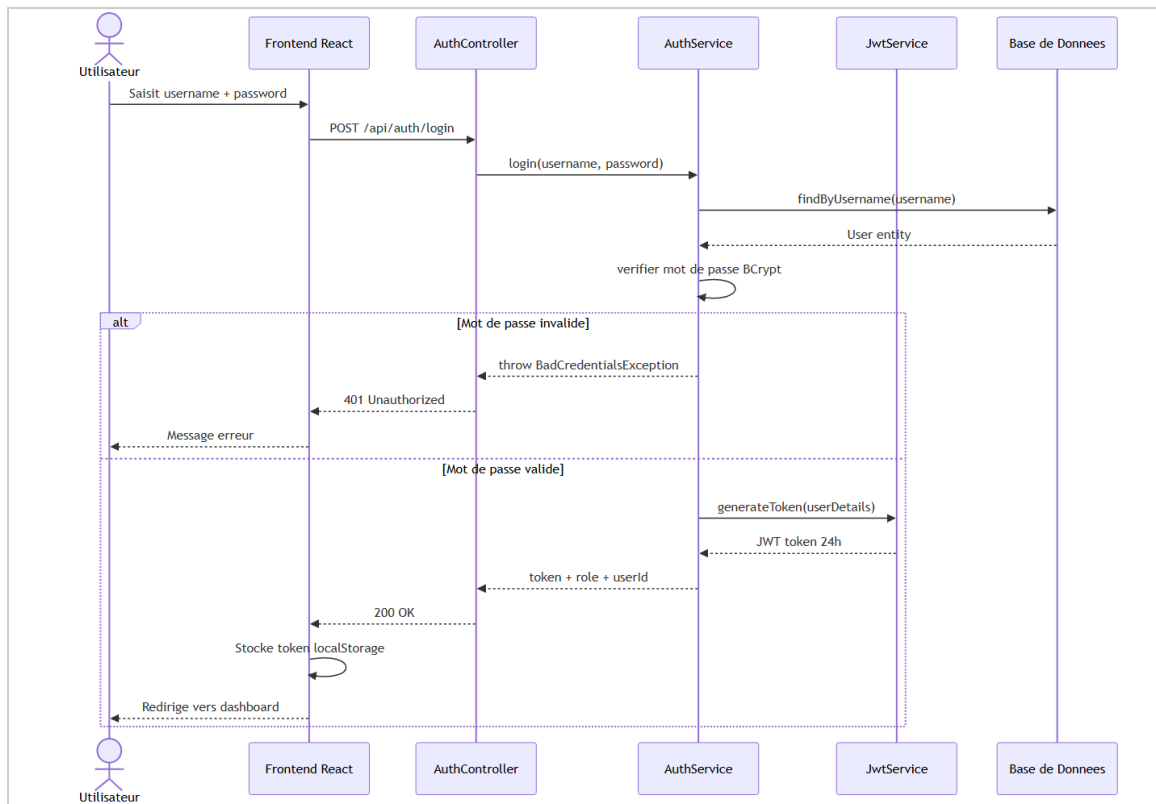


Figure 2-16 - Diagramme de sequence : Authentification

- Diagramme de sequence du processus d'execution :

Cette Figure 2-17 represente le flux complet d'un cycle de test. Le testeur cree une session de test (statut OUVORTE), lance l'execution d'un cas de test (statut PENDING), met a jour le statut apres execution (ici FAILED), puis declare une anomalie critique. Le systeme enregistre l'anomalie avec le statut OUVORTE et declenche automatiquement des notifications aux Admin et Manager via le NotificationService.

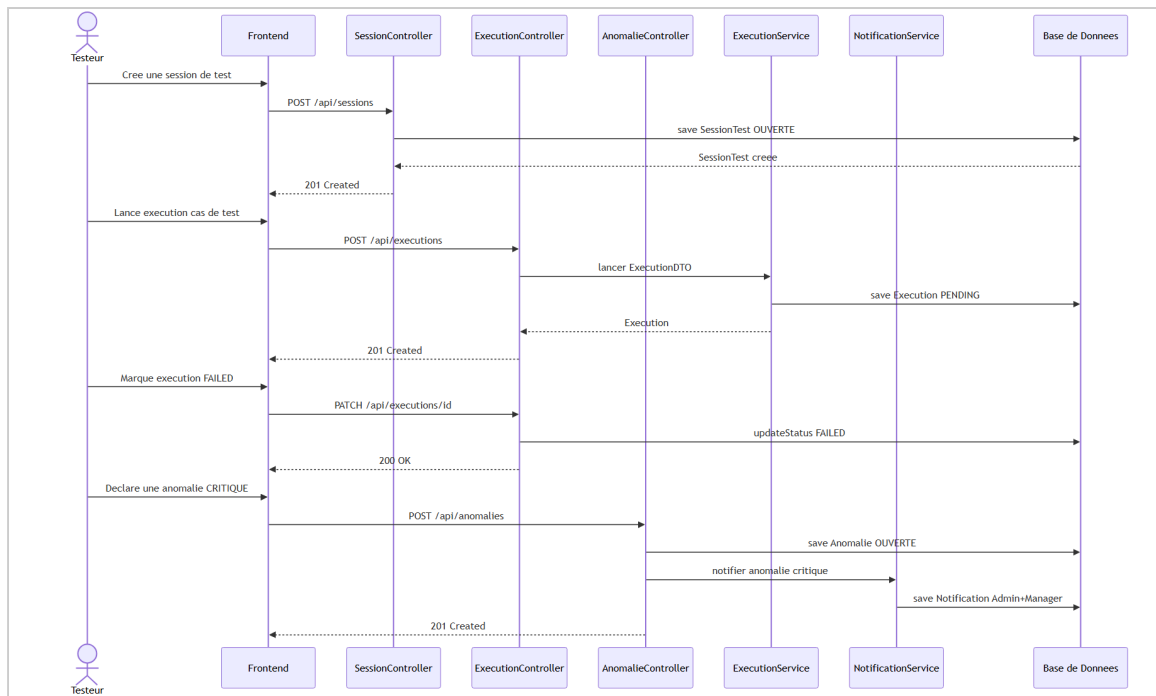


Figure 2-17 - Diagramme de sequence : Processus d'execution de test

- Diagramme de sequence de generation de cas de test par IA :

Cette Figure 2-18 represente le processus de generation automatique de cas de test via l'intelligence artificielle. Le testeur saisit une description fonctionnelle et le nombre de cas souhaite. Le AiCasTestService recupere le contexte du scenario, construit un prompt et l'envoi au serveur Ollama. Le LLM renvoie une reponse JSON structuree. Le service parse la reponse et sauvegarde chaque cas de test en base de donnees avec le flag genereParIA=true.

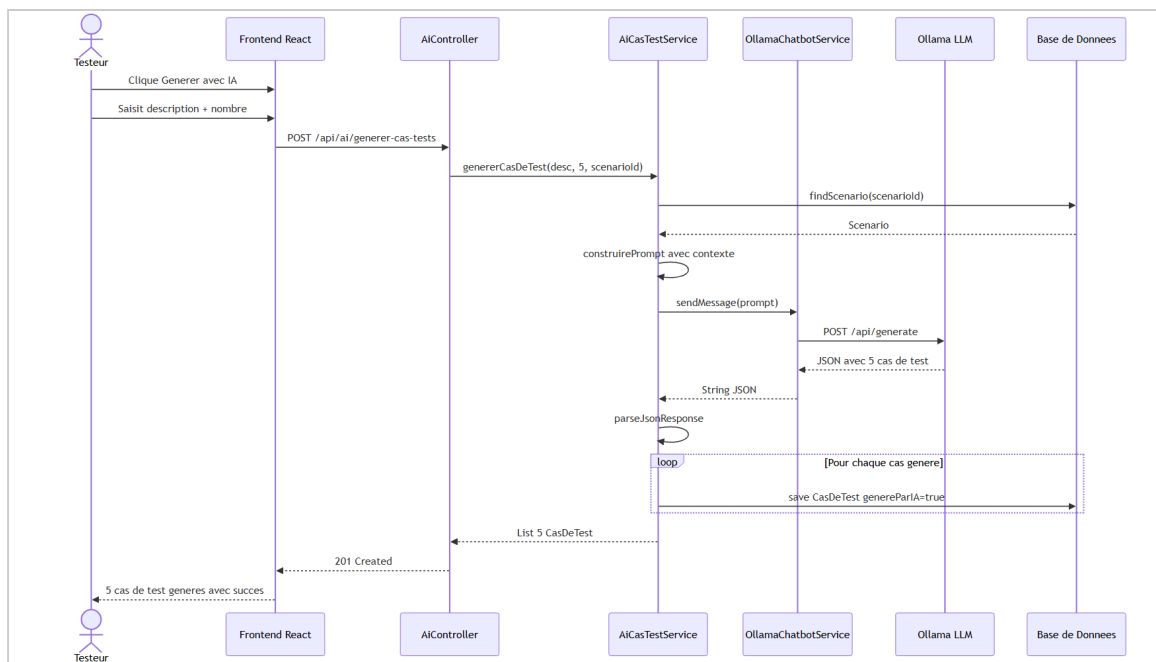


Figure 2-18 - Diagramme de sequence : Generation de cas de test par IA

- Diagramme de sequence de resume intelligent de session :

Cette Figure 2-19 represente le processus de generation automatique d'un resume de session lors de sa cloture. Le SessionTestService recupere toutes les executions et anomalies de la session. Le SessionSummaryService calcule les statistiques (taux de succes, anomalies par gravite) et envoie un prompt au LLM. Ollama genere un resume en francais avec une recommandation go/no-go. Le resume est sauvegarde et le statut de la session passe a CLOTUREE.

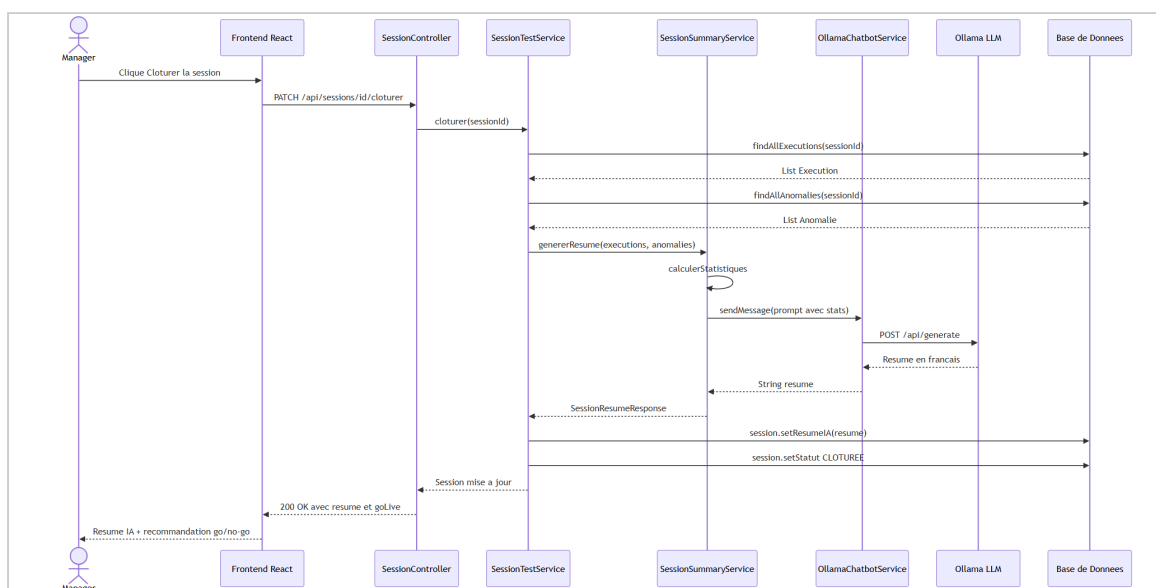


Figure 2-19 - Diagramme de sequence : Resume intelligent de session par IA

- Diagramme de sequence de suggestion de gravite par IA :

Cette Figure 2-20 represente le processus de suggestion automatique de gravite d'une anomalie. Lors de la saisie du titre et de la description, apres un debounce de 800ms, le frontend envoie une requete au AnomalieGraviteService. Celui-ci envoie un prompt au LLM qui analyse le contenu et renvoie une suggestion (par exemple "CRITIQUE - perte de donnees") avec un indice de confiance (92%). Le testeur peut accepter ou modifier la suggestion avant de soumettre le formulaire.

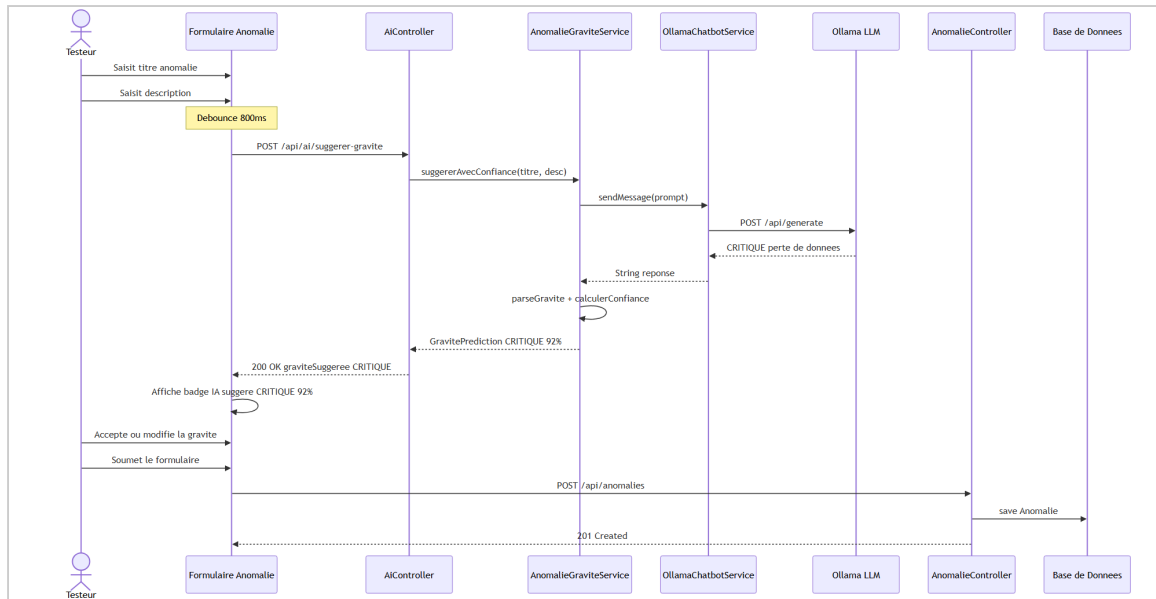


Figure 2-20 - Diagramme de sequence : Suggestion de gravite par IA

2.4.3 Diagramme de classes

Le diagramme de classes est l'un des diagrammes les plus populaires en langage UML. Très utilisé par les ingénieurs logiciels pour documenter l'architecture de leurs logiciels, les diagrammes de classes sont un type de diagramme de structure, car ils décrivent ce qui doit être présent dans le système modélisé. La forme de la classe se compose d'un rectangle à trois lignes. La ligne supérieure contient le nom de la classe, celle du milieu affiche les attributs et la ligne inférieure exprime les méthodes ou opérations.

Vue Statique

La vue statique de l'application comprend différents composants qui interagissent pour fournir les fonctionnalités attendues. En utilisant le langage UML, nous avons modélisé les données et structures de données nécessaires. Le diagramme de classes illustre les relations entre les différentes entités telles que les utilisateurs, les projets, les modules, les scénarios, les cas de test, les exécutions, les anomalies et les sessions de test.

- Description des composants de la vue statique

Voir Tableau 2-2 : ce tableau présente la description de chaque classe utilisée dans le diagramme de classes.

Classe	Description	Attributs principaux
Utilisateur (User)	Classe mere representant un utilisateur du systeme. Heritee par Admin, Manager et Testeur via la strategie SINGLE_TABLE.	id, username, email, password, nom, prenom, role, actif, dateCreation
Admin	Specialisation de Utilisateur avec le role ADMIN. Gere les utilisateurs, les projets et les archives.	Herite de Utilisateur
Manager	Specialisation de Utilisateur avec le role MANAGER. Valide les projets et supervise le processus de test.	Herite de Utilisateur
Testeur	Specialisation de Utilisateur avec le role TESTEUR. Execute les tests et declare les anomalies.	Herite de Utilisateur
Projet	Represente un projet de test logiciel. Contient des modules et est associe a des membres.	id, nom, description, dateDebut, dateFin, statut (EN_COURS, TERMINE, VALIDE, ARCHIVE)
ModuleProjet	Represente un module fonctionnel au sein d'un projet (ex: Module Authentification).	id, nom, description, projet
Scenario	Represente un parcours fonctionnel a tester au sein d'un module.	id, titre, description, module
CasDeTest	Decrit une procedure precise de test avec donnees d'entree, etapes et resultat attendu.	id, titre, description, etapes, donneesEntree, resultatAttendu, genereParIA, scenario
SessionTest	Represente une campagne de test regroupant des executions dans un contexte temporel.	id, nom, dateCreation, statut (OUVERTE, CLOTUREE), commentaire, projet, testeur

Execution	Représente l'exécution concrète d'un cas de test dans une session.	id, dateExecution, statut (PENDING, SUCCESS, FAILED, BLOCKED), captureUrl, casDeTest, sessionTest
Anomalie	Représente un défaut détecté lors d'une exécution échouée.	id, titre, description, gravité (FAIBLE, MOYENNE, CRITIQUE), statut (OUVERTE, EN_COURS, CORRIGÉE, FERMÉE), execution
Commentaire	Commentaire attaché à une anomalie pour le suivi collaboratif.	id, contenu, dateCreation, utilisateur, anomalie
Notification	Alerte envoyée automatiquement à un utilisateur (ex: anomalie critique).	id, message, dateCreation, lue, utilisateur

Tableau 2-2 - Description des composants de la vue statique

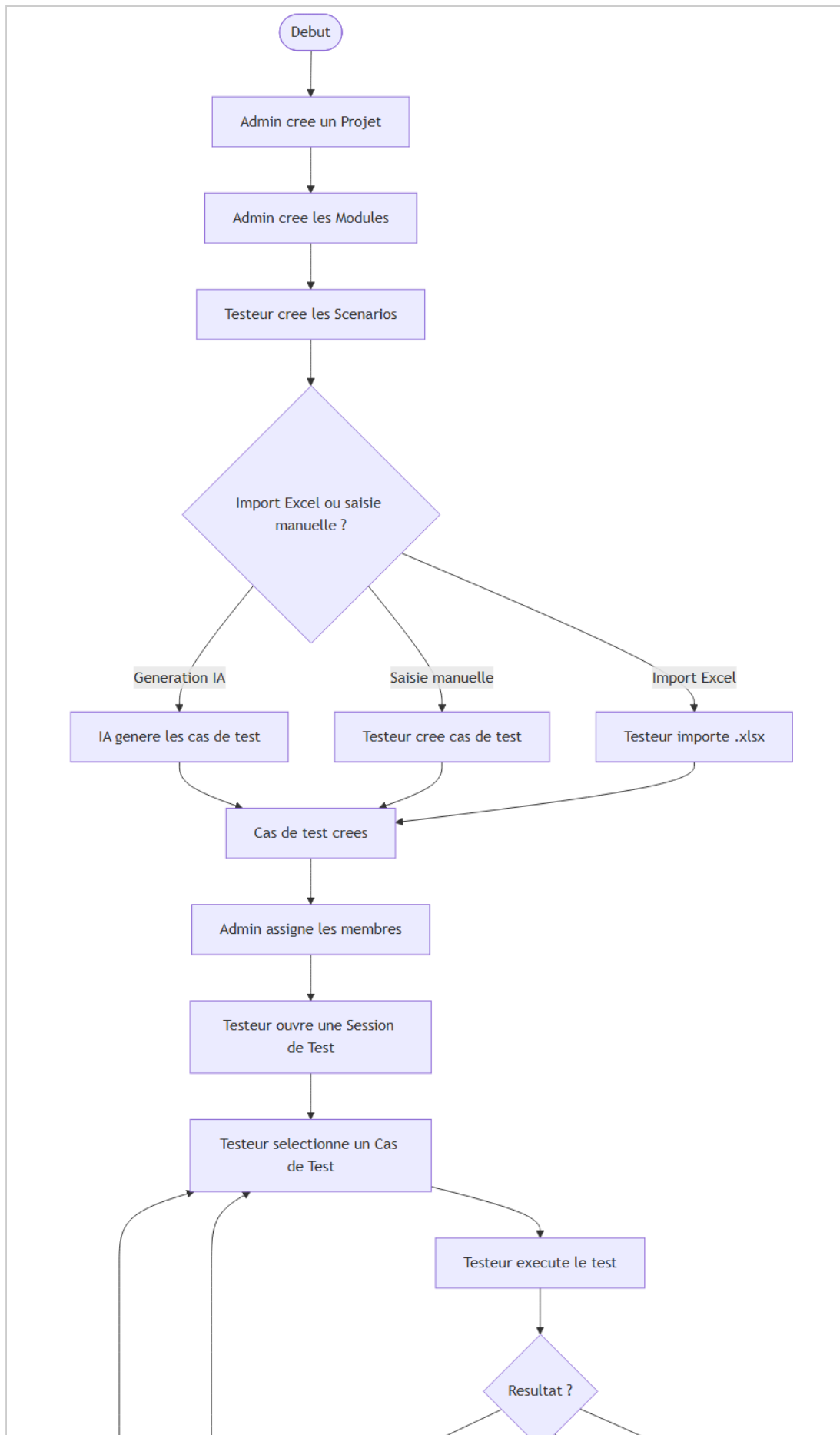
- Diagramme de classes pour la vue statique

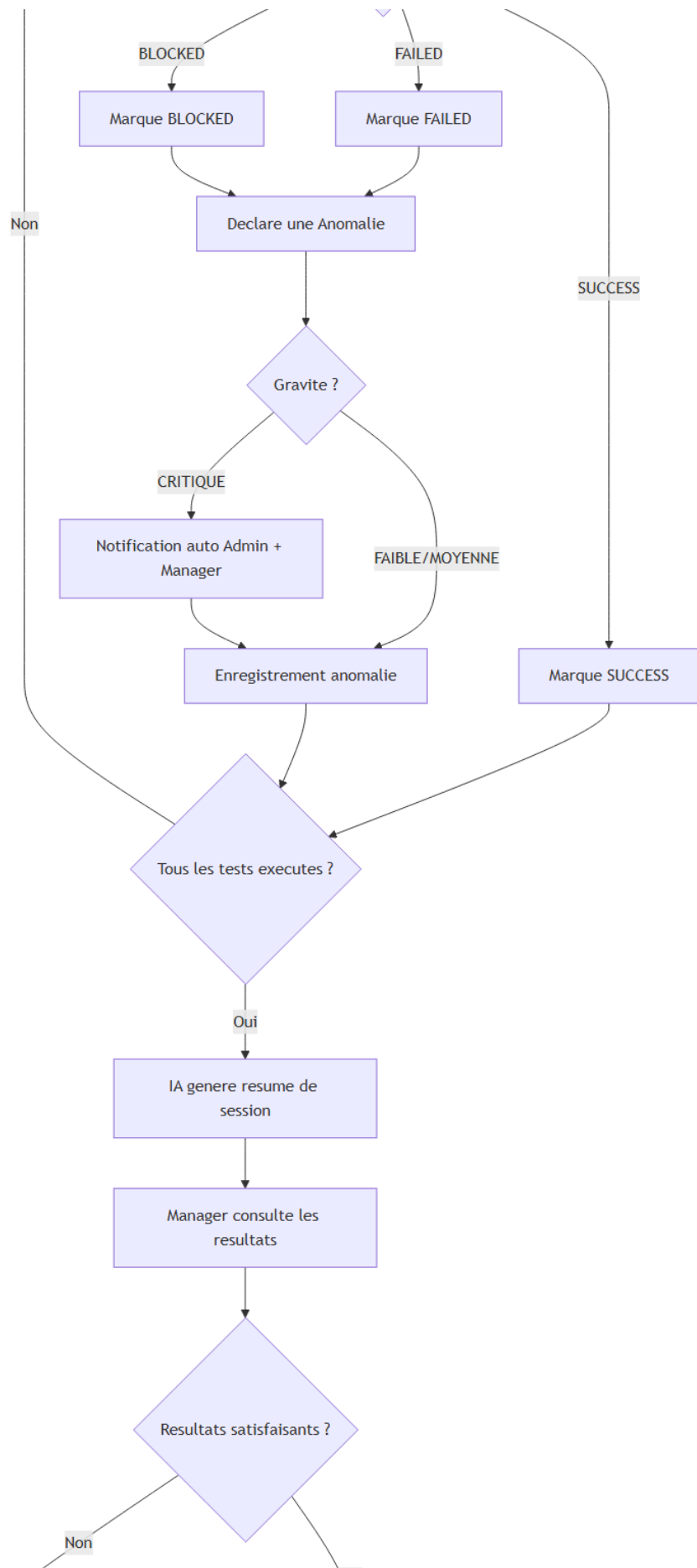
Dans cette partie, nous présentons le diagramme de classes spécifique à la vue statique de notre application Test Management Platform. Ce diagramme offre une représentation visuelle des classes et de leurs relations. La hiérarchie d'héritage (Utilisateur → Admin/Manager/Testeur) est implémentée via la stratégie JPA SINGLE_TABLE. Les multiplicités indiquent les cardinalités des associations (1..*, 0..*, 0..1).

- **Flux de données unidirectionnel** : React suit un flux de données unidirectionnel. Les données se propagent du state vers les composants via les props et les contextes.
- **Communication API** : les composants communiquent avec le backend via les services Axios. Chaque requête inclut automatiquement le token JWT dans l'en-tête Authorization.

2.4.4 Diagramme d'activite

Le diagramme d'activite modelise le flux de travail global du processus de gestion de test, depuis la creation d'un projet jusqu'a son archivage. Il met en evidence les points de decision et les chemins alternatifs. Le processus se deroule en sept phases : initialisation du projet par l'Admin, preparation des cas de test (saisie manuelle, import Excel ou generation IA), assignation des membres, execution des tests par le Testeur, gestion des anomalies avec notifications automatiques, bilan avec resume IA, et enfin validation par le Manager suivie de l'archivage.





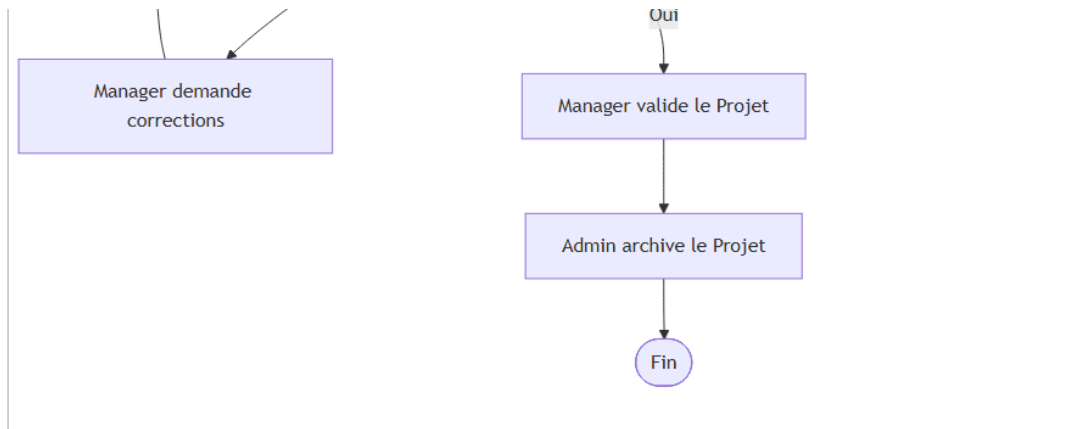


Figure 2-22 - Diagramme d'activite : Processus global de gestion de test

2.4.5 Diagrammes d'etats-transitions

Les diagrammes d'etats-transitions modelisent les differents etats possibles d'une entite et les evenements qui provoquent les transitions entre ces etats.

- Diagramme d'etats-transitions d'un projet :

Cette Figure 2-23 represente le cycle de vie d'un projet. Un projet est cree par l'Admin avec le statut EN_COURS, puis passe a TERMINE lorsque les tests sont executes. Le Manager peut le valider (statut VALIDE) ou le refuser (retour a EN_COURS). L'Admin peut archiver le projet a tout moment (statut ARCHIVE, etat terminal).

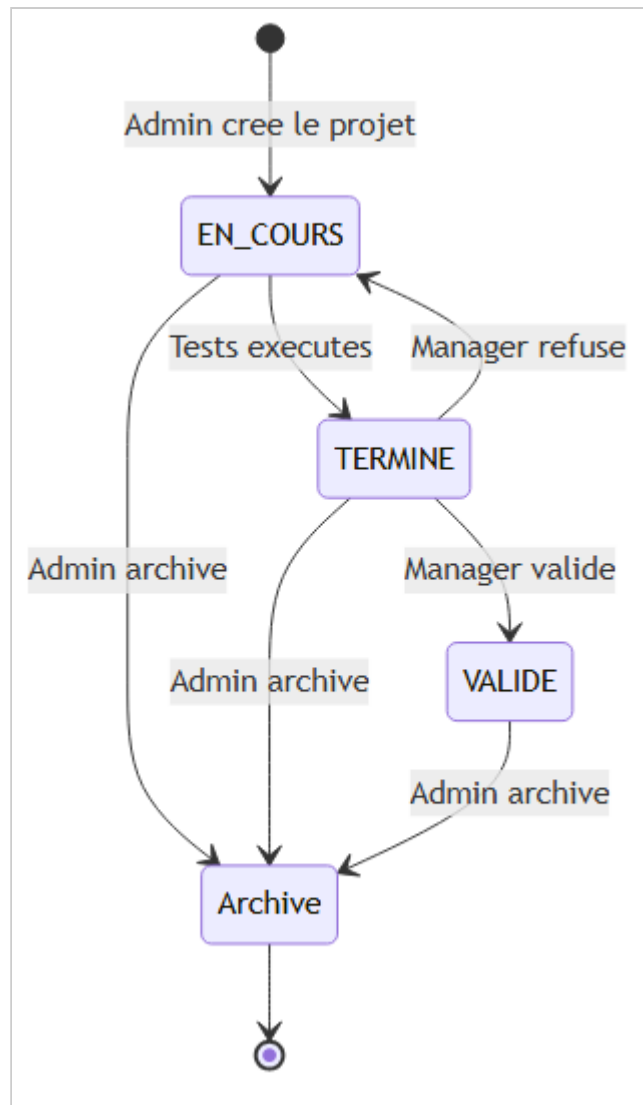


Figure 2-23 - Diagramme d'etats-transitions : Projet

- Diagramme d'etats-transitions d'une anomalie :

Cette Figure 2-24 represente le cycle de vie d'une anomalie. Elle est declaree avec le statut OUVERTE, passe a EN_COURS lors de la prise en charge, puis a CORRIGEE apres correction. Le Manager valide la correction (FERMEE) ou la renvoie (retour a EN_COURS). Un faux positif peut etre ferme directement.

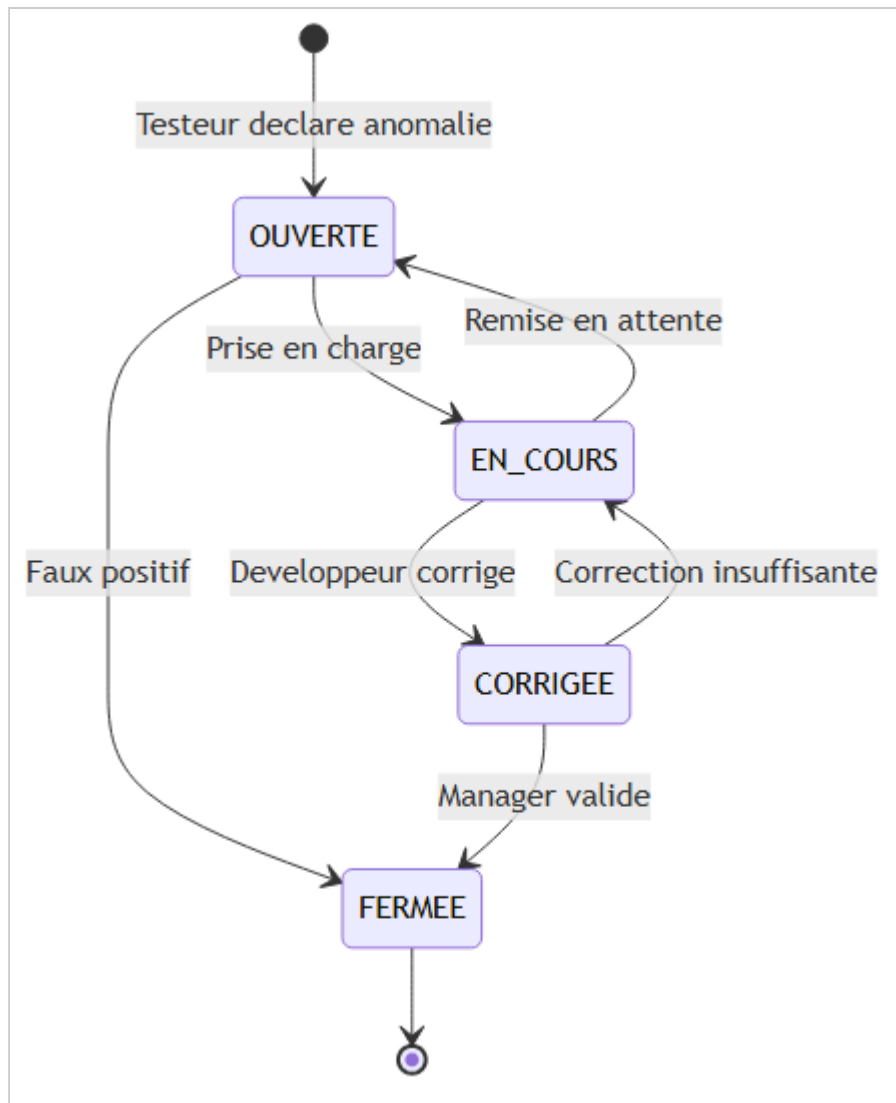


Figure 2-24 - Diagramme d'etats-transitions : Anomalie

- Diagramme d'etats-transitions d'une execution :

Cette Figure 2-25 represente le cycle de vie d'une execution. Elle demarre avec le statut PENDING, puis passe a SUCCESS, FAILED ou BLOCKED. Des transitions bidirectionnelles sont possibles entre SUCCESS et FAILED (re-execution). Un test BLOCKED peut revenir a PENDING lorsque le prerequis est resolu. Toutes les executions atteignent leur etat final lors de la cloture de la session.

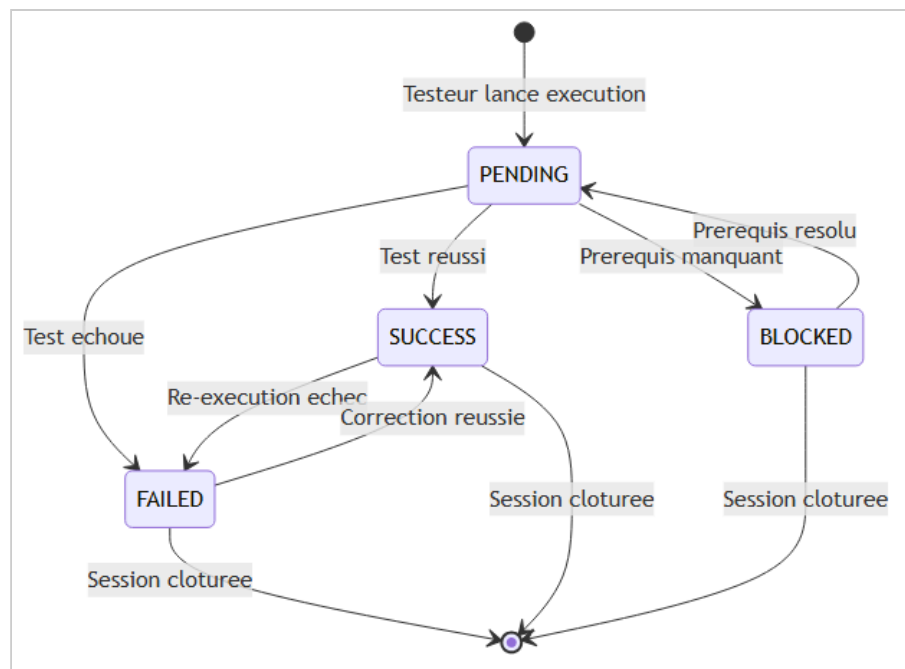


Figure 2-25 - Diagramme d'etats-transitions : Execution

2.4.6 Diagramme de composants

Le diagramme de composants (Figure 2-26) modelise l'architecture logicielle du systeme. Le frontend React (port 5173) comprend les composants IA (GenerateurCasTestModal, GraviteBadgeIA, SessionResumePanel), les pages par role et les services API Axios. Le backend Spring Boot (port 8081) inclut la couche Spring Security JWT, les controleurs REST (AuthController, ProjetController, ExecutionController, AnomalieController, AiController) et les services metier. Les services externes comprennent MySQL 8 pour la persistance, Cloudinary CDN pour les images et Ollama LLM (port 11434) pour l'intelligence artificielle.

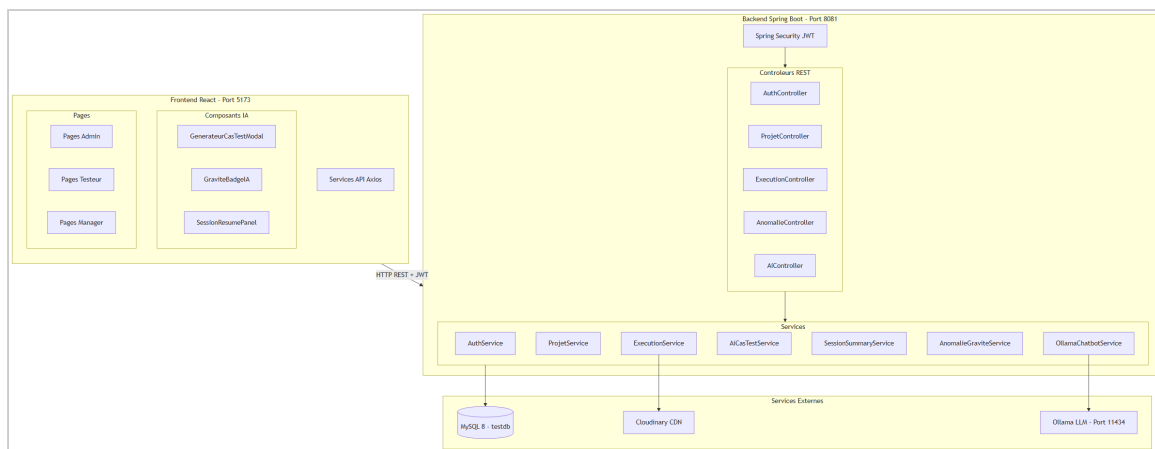


Figure 2-26 - Diagramme de composants

2.4.7 Diagramme de déploiement

Le diagramme de déploiement (Figure 2-27) represente l'infrastructure physique du systeme. Le navigateur client communique avec le frontend React via HTTP (port 5173). Le frontend communique avec le backend Spring Boot via HTTP REST + JWT (port 8081). Le backend accede a la base MySQL via JDBC (port 3306), au serveur Ollama via HTTP (port 11434) et a Cloudinary via HTTPS. Tous les composants sont deployes sur une meme machine serveur, a l'exception de Cloudinary qui est un service cloud.

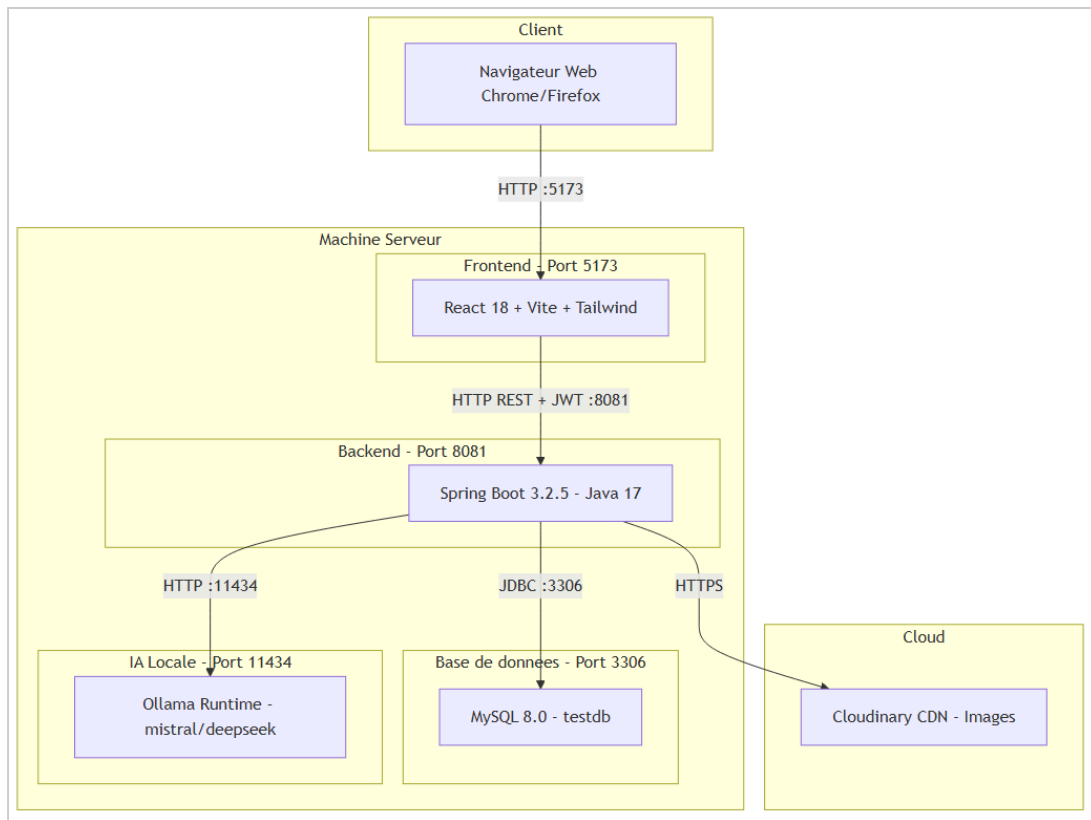


Figure 2-27 - Diagramme de déploiement

2.5 Conclusion

En resume, ce chapitre a etabli les besoins fonctionnels et non fonctionnels essentiels de l'application, tout en presentant son architecture a travers les diagrammes UML. Nous avons identifie trois acteurs principaux (Admin, Manager, Testeur) avec des permissions clairement differenciees, et modelise l'ensemble des interactions via 27 diagrammes UML couvrant les perspectives statique, dynamique et physique du systeme.

Les diagrammes de cas d'utilisation ont permis de formaliser les fonctionnalites accessibles par chaque role. Les diagrammes de sequence ont illustre les flux d'interaction pour les scenarios les plus critiques, incluant les trois fonctionnalites d'intelligence artificielle (generation de cas de test, resume de session, suggestion de gravite). Le diagramme de classes a modelise la structure de donnees avec la hierarchie d'heritage et les associations entre entites. Les diagrammes complementaires (activite, etats-transitions, composants, deploiement) ont complete la vision globale de l'architecture.

Cette synthese fournit une base solide pour le developpement et constitue le socle technique indispensable a la realisation de la plateforme Test Management Platform, presentee dans le chapitre suivant.